

AIRBNB DATABASE IMPLEMENTATION

Implementation Overview

This phase focused on transforming the conceptual ER model from Phase 1 into a fully functional MySQL database for an Airbnb-style property rental platform. The implementation followed a systematic approach to ensure data integrity, correct relationships, and comprehensive business logic.

Database Structure

The database comprises 22 normalized tables that cover all aspects of the platform. These tables include:

- User Management: User, Role
- Property Listings: Property, Location, PropertyType, Amenity
- Booking Transactions: Booking, BookingStatus
- Payment Processing: Payment, PaymentMethod, PaymentStatus, Payout
- User Interactions: Review, Message
- System Management: AuditLog, Dispute, Verification, HostReport

Each table has appropriate primary keys, foreign key constraints, and indexes to ensure optimal performance.

Key Relationships Implemented

I successfully established three essential relationships:

- the Booking relationship, which connects the Guest (User) to Property and Booking Status;
- the Payment relationship, linking Booking with Payment Method and Payment Status; and
- the Review relationship, which connects Booking to both Reviewer and Reviewee (both of which are User entities).

Furthermore, I implemented two recursive relationships: Message, enabling communication between users, and Review, allowing users to evaluate one another.

Sample Data Population

Over 20+ sample records were inserted into each main table, resulting in 24 users, 22 properties, 22 bookings, 28 amenities, 12 messages, and supporting data across all tables. The insert order carefully respected foreign key dependencies to maintain referential integrity.

Advanced Database Features

The implementation includes 3 views for simplified data access (vw_property_listings, vw_booking_details, vw_host_dashboard), 2 stored procedures for business operations (sp_make_booking, sp_process_payment), 3 triggers for automated processes, and 2 custom functions for reusable calculations. Performance indexes were added to optimize query execution.

Testing and Validation

All relationships were validated through comprehensive test queries, confirming that triple and recursive relationships function correctly and that business rules (like preventing double bookings) are enforced at the database level.

Total Objects Created: 22 Tables, 3 Views, 2 Procedures, 3 Triggers, 2 Functions